

EPFL CIVIL-127, Lab 4 solution

4.1

```
# Solution for 4.1
# See https://github.com/vita-epfl/civil127-2026/blob/main/lab4/lab4.pdf

import math

def rope_length(x: float) -> float:
    """
    Calculate rope length as a function of x.

    alpha = PMA angle
    beta = AMB angle
    e = PA length
    """
    alpha = math.acos(1/x)
    beta = 2 * math.pi - math.pi/2 - 2 * alpha
    e = math.sqrt(x*x - 1)
    # since our angle is in radians and the radius 1, beta is also the arc length
    return 2 * (e + beta + 1)

def solve_x(L: float, accuracy: float) -> float:
    """
    Find a value for x using binary search. We are assuming rope_length() is
    monotonic from sqrt(2) onwards.
    """
    low = math.sqrt(2)
    high = L

    while True:
        mid = (low + high) / 2
        t = rope_length(mid)
        if abs(t - L) < accuracy:
            # stop when we are +/- accuracy with the desired length
            return mid
        if t > L:
            high = mid
        else:
            low = mid
```

```
rope = (math.pi * 2 + 4) * 1.000001
x = solve_x(rope, 0.00001)
print("x=", x)
```

If my code is correct, the answer is $x=1.416\dots$. This problem was inspired by a popular “rope around the world” math puzzle. An interesting fact in both cases is that adding a little bit to the rope causes the rope to rise by a lot more.

4.2

Part 1

```
# Battleship generator (4.2, part 1)
# Uses a dict for the grid
# see https://github.com/vita-epfl/civil127-2026/blob/main/lab4/lab4.pdf

import random

type coord = tuple[int, int]

class Battleship:
    """
    Represents one battleship board.
    For now, enables randomly placing ships on the grid and printing
    the grid.
    """

    def __init__(self, width: int, height: int):
        self.width = width
        self.height = height
        self.grid: dict[coord, str] = {}

    def will_fit(self, length: int, pos: coord, dir: coord) -> bool:
        """
        Checks if a ship of given length will fit in the grid.

        dir must be either (1, 0) for horizontal ships or (0, 1) for vertical
        ships.
        """
```

```

x, y = pos
dx, dy = dir
for i in range(length):
    if (x, y) in self.grid:
        # There's already a ship here
        return False
    if x == self.width or y == self.height:
        # We are going out of bounds
        return False
    x += dx
    y += dy
return True

def place(self, letter: str, length: int, pos: coord, dir: coord) -> None:
    """
    Places a ship on the grid. Assumes that the ship will fit.

    dir must be either (1, 0) for horizontal ships or (0, 1) for vertical
    ships.
    """
    x, y = pos
    dx, dy = dir
    for i in range(length):
        self.grid[(x, y)] = letter
        x += dx
        y += dy

def randomize_grid(self) -> None:
    """
    Initialize the grid by randomly placing the ships.
    """
    ships = {
        "a": (5, "carrier"),
        "b": (4, "battleship"),
        "c": (3, "cruiser"),
        "d": (3, "submarine"),
        "e": (2, "destroyer"),
    }
    for ship, (length, _) in ships.items():
        ok = False
        while not ok:
            x = random.randint(0, self.width - 1)
            y = random.randint(0, self.height - 1)

```

```

        pos = (x, y)
        dir = random.choice([(1, 0), (0, 1)])
        if self.will_fit(length, pos, dir):
            self.place(ship, length, pos, dir)
            ok = True

def print_grid_with_ships(self):
    """
    Prints the grid with the ships, useful for debugging
    """

    # print top header
    print(" ", end=" ")
    for i in range(0, self.width):
        print("ABCDEFGHJKLMN"[i], end=" ")
    print()

    for y in range(0, self.height):
        # print letter for each row
        print(y + 1, end=" ")
        for x in range(0, self.width):
            if (x, y) in self.grid:
                print(self.grid[x, y], end=" ")
            else:
                print(".", end=" ")
        print()

random.seed(94)
game = Battleship(13, 9)
game.randomize_grid()
game.print_grid_with_ships()

```

Part 2

If you solved part 2, share your solution on the forum!

```

# Battleship generator (4.2, part 1+2)
# Uses a dict for the grid
# see https://github.com/vita-epfl/civil127-2026/blob/main/lab4/lab4.pdf

```

```

import random

type coord = tuple[int, int]

class Battleship:
    """
    Represents one battleship board.
    Enables randomly placing ships on the grid and printing
    the grid.
    Enables one-player play.
    """

    COLS = "ABCDEFGHJKLM" # constant defined at the class level
    ROWS = "123456789"

    def __init__(self, width: int, height: int):
        self.width = width
        self.height = height
        self.grid: dict[coord, str] = {}
        self.shots: set[coord] = set()

    def will_fit(self, length: int, pos: coord, dir: coord) -> bool:
        """
        Checks if a ship of given length will fit in the grid.

        dir must be either (1, 0) for horizontal ships or (0, 1) for vertical
        ships.
        """
        x, y = pos
        dx, dy = dir
        for i in range(length):
            if (x, y) in self.grid:
                # There's already a ship here
                return False
            if x == self.width or y == self.height:
                # We are going out of bounds
                return False
            x += dx
            y += dy
        return True

    def place(self, letter: str, length: int, pos: coord, dir: coord) -> None:

```

```

"""
Places a ship on the grid. Assumes that the ship will fit.

dir must be either (1, 0) for horizontal ships or (0, 1) for vertical
ships.
"""
x, y = pos
dx, dy = dir
for i in range(length):
    self.grid[(x, y)] = letter
    x += dx
    y += dy

def randomize_grid(self) -> None:
    """
    Initialize the grid by randomly placing the ships.
    """
    self.ships = {
        "a": (5, "carrier"),
        "b": (4, "battleship"),
        "c": (3, "cruiser"),
        "d": (3, "submarine"),
        "e": (2, "destroyer"),
    }
    for ship, (length, _) in self.ships.items():
        ok = False
        while not ok:
            x = random.randint(0, self.width - 1)
            y = random.randint(0, self.height - 1)
            pos = (x, y)
            dir = random.choice([(1, 0), (0, 1)])
            if self.will_fit(length, pos, dir):
                self.place(ship, length, pos, dir)
                ok = True

def print_grid_with_ships(self):
    """
    Prints the grid with the ships, useful for debugging
    """

    # print top header
    print(" ", end=" ")
    for i in range(0, self.width):

```

```

        print(Battleship.COLS[i], end=" ")
    print()

    for y in range(0, self.height):
        # print letter for each row
        print(y + 1, end=" ")
        for x in range(0, self.width):
            if (x, y) in self.grid:
                print(self.grid[x, y], end=" ")
            else:
                print(".", end=" ")
        print()

def print_grid_with_shots(self):
    """
    Prints the grid with the shots

    Note: there's some code duplication between the two printing methods.
    It might make sense to merge them or refactor the common parts.
    """

    print("\033c") # clear screen

    # print top header
    print(" ", end=" ")
    for i in range(0, self.width):
        print(Battleship.COLS[i], end=" ")
    print()

    for y in range(0, self.height):
        # print letter for each row
        print(y+1, end=" ")
        for x in range(0, self.width):
            if (x, y) in self.shots:
                if (x, y) in self.grid:
                    print("#", end=" ")
                else:
                    print("~", end=" ")
            else:
                print(".", end=" ")
        print()
    print()

```

```

def play(self) -> None:
    """
    Game loop. Prompts for input (in the form of <A-M letter><1-9 number>)
    until all the ships have been sunk.
    Assumes the grid has been initialized.
    """

    # Track which ships are left. Each ship starts with an integer equal to
    # its length. As the ship gets hit, the integer is decremented. When
    # the ship is sunk, it is removed from this data structure.
    # note: we assume you can't shoot in the same space twice.
    ships_hit: dict[str, int] = {}
    for ship, (length, _) in self.ships.items():
        ships_hit[ship] = length

    self.print_grid_with_shots()
    while len(ships_hit) > 0:
        s = input("target? ")

        # Check the input is valid
        if len(s) != 2:
            print("what?")
            continue
        if s[0] not in Battleship.COLS:
            print("what?")
            continue
        x = Battleship.COLS.index(s[0])

        if s[1] not in Battleship.ROWS:
            print("what?")
            continue
        y = Battleship.ROWS.index(s[1])

        if (x, y) in self.shots:
            print("You already fired here")
            continue

        # When we reach here, the (x, y) coordinates are valid
        self.shots.add((x, y))
        self.print_grid_with_shots()
        if (x, y) in self.grid:
            # we hit a ship
            ship = self.grid[(x, y)]

```



```
        ships_hit[ship] -= 1
        length, ship_name = self.ships[ship]
        if ships_hit[ship] == 0:
            print("You sunk my", ship_name)
            del ships_hit[ship]
        else:
            print("You hit my", ship_name, "(", length, ")")
    else:
        print("Miss")
    print()
    print("You sunk all the things!")

game = Battleship(13, 9)
game.randomize_grid()
game.play()
```